

Secure Backend Architecture & APIs

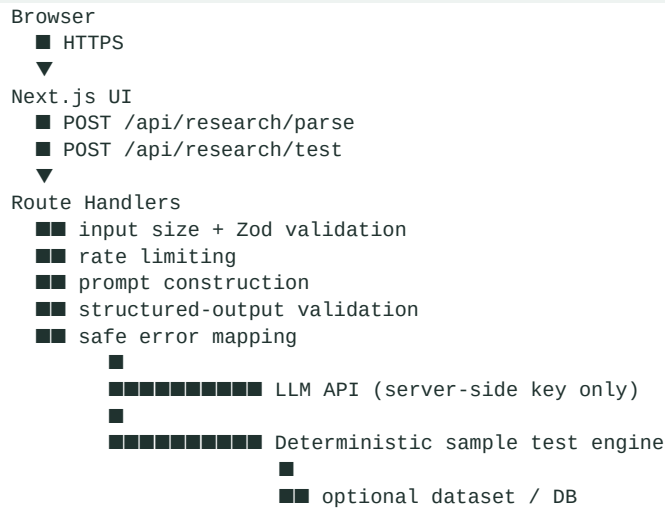
Minimal full-stack architecture, API contracts, validation, security controls, storage choices, and third-party services.

Recommended build: Option 2 - ASK -> CLARIFY -> DEFINE -> TEST -> LEARN

Source basis: the complete 13-page assignment PDF provided by you. Page references inside these guides refer to that assignment.

1. Secure minimal full-stack architecture

Recommended stack: Next.js + TypeScript + Tailwind/shadcn, server-side route handlers, Zod validation, one LLM provider, and optional persistence. Keep keys and model calls on the server.



Why this architecture

- Small enough for the stated 3-4 hour assignment scope.
- Separates AI parsing from experiment testing.
- Prevents browser exposure of API keys.
- Allows the parser, model or test engine to be replaced later.
- Makes it easy to explain responsibilities during evaluation.

2. API contracts

POST /api/research/parse

Request

```
{ "query": "Does buying NIFTY after a sharp fall work?",
  "clarifications": { "holdingPeriodDays": 5 } }
```

Response

```
{
  "experiment": { ...validated Experiment },
  "missingFields": [ ... ],
  "assumptions": [ ... ],
  "readyToTest": false
}
```

POST /api/research/test

Request

```
{ "experiment": { ...complete Experiment } }
```

Response

```
{
  "method": "simulated" | "sample-dataset",
  "metrics": { "trades": 42, "meanReturn": 0.006, ... },
  "evidence": [ ... ],
  "warnings": [ ... ]
}
```

3. Security checklist

Risk	Implementation
API key leak	Environment variables on server only; never NEXT_PUBLIC_* for secrets.
Prompt injection	Treat user question as data; fixed system instructions; never let user control tool/system messages.
Malformed AI output	Structured output + Zod parse; retry once or return safe validation error.
Abuse / cost explosion	Input length cap + IP/session rate limit + model max output tokens.
XSS	Render model text as plain React text; no dangerouslySetInnerHTML.
Oversized payload	Set JSON/body limits and query length (e.g. 1-2k chars).
DoS via repeated clicks	Disable during pending request; server rate limit.
Sensitive logging	Do not log API keys or raw headers; redact provider errors.
CORS	Same-origin routes for prototype; explicit CORS only if separated frontend/backend.
Data trust	Label mock/sample data prominently; never call it live market data.

Validation schema sketch

```
const ExperimentSchema = z.object({
  instrument: z.string().min(1),
  timeframe: z.enum(['daily', 'hourly']).default('daily'),
  entry: z.object({ type: z.string(), thresholdPct: z.number().optional() }),
  exit: z.object({ type: z.string(), value: z.number().optional() }).nullable(),
  holdingPeriodDays: z.number().int().min(1).max(252).nullable(),
  testPeriod: z.object({ start: z.string(), end: z.string() }).nullable(),
  filters: z.array(z.object({ name: z.string(), value: z.string() })),
  costs: z.object({ transactionBps: z.number(), slippageBps: z.number() }),
  hypothesis: z.string().min(1),
  provenance: z.record(z.enum(['user', 'assumption', 'clarified', 'derived']))
});
```

4. Persistence: optional, not mandatory

The assignment allows any suitable database/storage but does not require one. A strong prototype can avoid persistence entirely. If you add history, use it to demonstrate “remember what it learned” or saved experiments - otherwise it is unnecessary scope.

Choice	When to use	Recommendation
No DB	Single-session prototype	Best default
localStorage	Keep last few experiments on same browser	Simple optional enhancement
Neon/Postgres + Drizzle	You want persistent experiment history across sessions	Only if core flow is already complete

5. Environment variables

```
# .env.local (never commit)
AI_PROVIDER=openai      # or gemini
```

```

OPENAI_API_KEY=...
# GEMINI_API_KEY=...

# Optional persistence
DATABASE_URL=...

# Optional rate-limit vendor
UPSTASH_REDIS_REST_URL=...
UPSTASH_REDIS_REST_TOKEN=...

```

Third-party/API recommendation

Tool/API	Role	Required?
OpenAI API or Gemini API	Question -> structured experiment + cautious interpretation	Use exactly one provider
Zod	Runtime validation	Yes
shadcn/ui / Radix	Accessible UI primitives	Optional but useful
Recharts	Small result visualization	Optional
Neon + Drizzle	Persist experiments	Optional
Upstash Redis	Distributed rate limiting	Optional for deployed prototype
Vercel	Deployment	Recommended if using Next.js

Do not call Alpha Vantage, Zerodha, Upstox, NSE scraping, or a brokerage API just to appear “real”. Option 2 explicitly permits sample/public/simulated data and does not require a production backtester.

Confirmed Implementation Stack

This page supersedes any earlier wording that presented the AI provider or database as optional choices. For this submission, the selected implementation is fixed:

Layer	Selected technology	How it is used
AI / LLM	Google Gemini API	Parse natural-language research questions, identify ambiguity, propose clarification questions
Database	Neon PostgreSQL	Persist research sessions, structured experiments, clarification answers, test runs, and con
ORM	Drizzle ORM	Typed schema, migrations, and database access from the Next.js server layer.
Validation	Zod	Validate both user input and Gemini structured output before persistence or testing.
Frontend / Backend	Next.js + TypeScript	App Router UI plus server-side route handlers/server functions.

Environment variables

```
GEMINI_API_KEY=...  
DATABASE_URL=postgresql://...  
GEMINI_MODEL=gemini-2.5-flash
```

Recommended data flow

Question -> server validation -> Gemini structured interpretation -> Zod validation -> clarification/confirmation -> experiment persisted to Neon -> deterministic TypeScript test -> result persisted -> Learn view.

Security rules

Never expose GEMINI_API_KEY or DATABASE_URL in client code. Keep both outside NEXT_PUBLIC variables. Validate request sizes, parse Gemini output against a strict schema, use parameterized ORM queries, add API rate limiting, and never let the LLM fabricate backtest metrics.

Suggested Neon schema

research_sessions: id, original_question, status, created_at
experiments: id, session_id, instrument, timeframe, entry_condition, exit_condition, holding_period, test_period, filters_json, costs_json, hypothesis, provenance_json
test_runs: id, experiment_id, dataset_source, metrics_json, warnings_json, created_at

Why this choice is strong for the assignment: Gemini is used for language understanding and ambiguity handling, while Neon stores reproducible research state. Numerical evidence remains deterministic and auditable instead of being invented by the model.